

信息学奥林匹克竞赛

Olympiad in Informatics

数据结构——线段树

陈 真

上海市实验学校

从RMQ问题谈起

【问题描述】有N个数，M次询问，每次给定区间[L,R]，求区间内的最大值。

【数据范围1】 $N \leq 10, M \leq 10$

【做法分析】暴力枚举 $O(N)$

【数据范围2】 $N \leq 10^5, M \leq 10^5$

【做法分析】线段树 $O(\log N)$ 处理每个询问

【数据范围3】 $N \leq 10^5, M \leq 10^6$

【做法分析】以上处理都不满足条件，需要 $O(1)$ 询问处理时间，需要用到**ST表**。

假设 $f(i, j)$ 为区间 $[i, j]$ 内的最大值，得转移方程

$$f(i, j) = \text{Max}(f(i, j-1), a_j)$$

这样操作预处理是 $O(N^2)$ ，是不行滴

Max操作允许区间重叠，也就是： $\text{Max}(a, b, c) = \text{Max}\{\text{Max}(a, b), \text{Max}(b, c)\}$

备注：ST表一般不能维护区间和

从RMQ问题谈起

【问题描述】有N个数，M次询问，每次给定区间[L,R]，求区间内的最大值。

【数据范围1】 $N \leq 10, M \leq 10$

【做法分析】暴力枚举 $O(N)$

【数据范围2】 $N \leq 10^5, M \leq 10^5$

【做法分析】线段树 $O(\log N)$ 处理每个询问

【数据范围3】 $N \leq 10^5, M \leq 10^6$

【做法分析】以上处理都不满足条件，需要 $O(1)$ 询问处理时间，需要用到**ST表**。

假设 $f(i, j)$ 为区间 $[i, j]$ 内的最大值，得转移方程

$$f(i, j) = \text{Max}(f(i, j-1), a_j)$$

这样操作预处理是 $O(N^2)$ ，是不行滴

Max操作允许区间重叠，也就是： $\text{Max}(a, b, c) = \text{Max}\{\text{Max}(a, b), \text{Max}(b, c)\}$

备注：ST表一般不能维护区间和

从RMQ问题谈起

利用倍增思想：ST表：一种利用 dp 思想求解区间最值的倍增算法

令 $f(i,j)$ 为从 a_i 开始的、连续 2^j 个数的最大值，显然：

$f(i,j)$ 表示 $[i, i + 2^j - 1]$ 这段长度为 2^j 的区间中的最大值

预处理： $f(i,0) = a_i$ 。即 $[i,i]$ 区间的最大值就是 a_i 。

```
for(int i=1;i<=N;i++) f[i][0]=a[i];
```

将区间 $[i, j]$ 平均分成两个区间：区间1： $[i, i + 2^{j-1} - 1]$

区间2： $[i + 2^{j-1}, i + 2^j - 1]$

两个区间的长度为 2^{j-1}

$f(i,j)$ 的最大值即这两段的最大值中的最大值。

$$f(i, j) = \max\{f(i, j-1), f(i + 2^{j-1}, j-1)\}$$

从RMQ问题谈起

$f(i,j)$ 表示 $[i, i + 2^j - 1]$ 这段长度为 2^j 的区间中的最大值

将区间 $[i, j]$ 平均分成两个区间：区间1： $[i, i + 2^{j-1} - 1]$

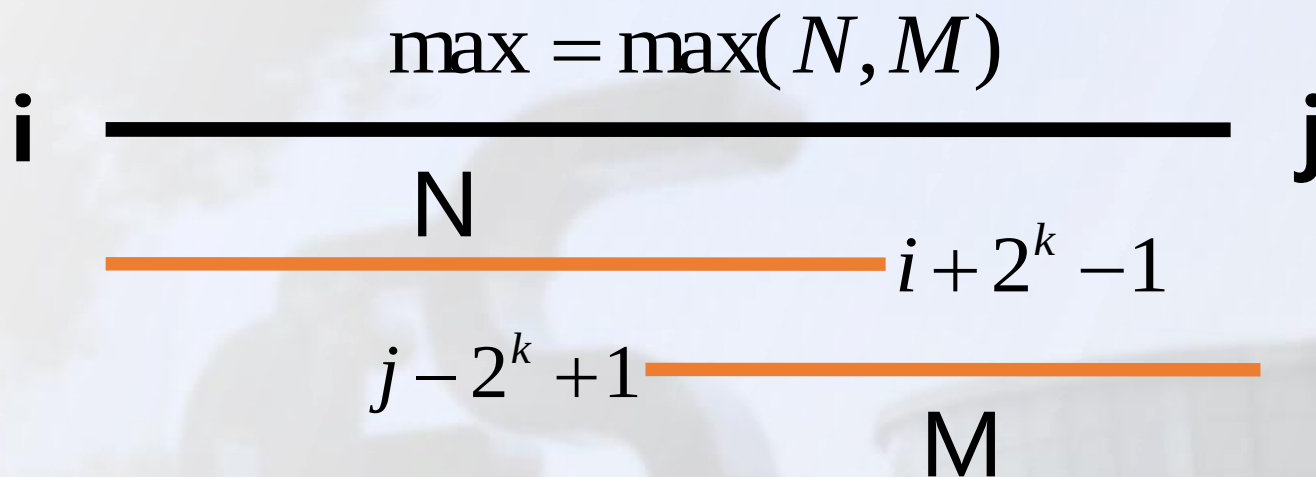
区间2： $[i + 2^{j-1}, i + 2^j - 1]$

$$f(i, j) = \max\{f(i, j-1), f(i + 2^{j-1}, j-1)\}$$

```
void RMQ(int N){  
    /*注意外部循环从j开始,  
    因为初始状态为f[i][0],  
    以i为外层会有一些状态遍历不到。*/  
    for(int j=1; j<=20; j++)  
        for(int i=1; i<=N; i++)  
            if(i+(1<<j)-1<=N)  
                f[i][j]=max(f[i][j-1], f[i+(1<<(j-1))][j-1]);  
}
```

从RMQ问题谈起

查询：若需要查询的区间为 $[i, j]$ ，则需要找到两个覆盖这个闭区间的最小幂区间。



区间长度为： $j-i+1$ ，可取 $k=\log_2(j-i+1)$

$$RMQ(i, j) = \max\{f(i, k), f(j - 2^k + 1, k)\}$$

从RMQ问题谈起

实例再现:

序号	1	2	3	4	5
ai	5	8	7	3	9

求解 $f(1,2)$ 也就是求解 $[1,4]$ 的最大值

1	2
5	8

3	4
7	3

将区间 $[1,4]$ 分为两个 $[1,2]$ 和 $[3,4]$ ， $f(1,1)$ 与 $f(3,1)$ ，而 $f(1,1)=8, f(3,1)=7$ ，则 $f(1,2)=\max(f(1,1), f(3,1))=8$

这种方式下，以每个点为起点都有 $O(\log N)$ 个区间，每个区间可以 $O(1)$ 求出，则预处理总时间、空间复杂度都为 $O(N \log N)$ 。

从RMQ问题谈起

那怎么处理询问呢?

根据MAX的性质, 可把区间拆成两个相重叠的区间, 如下图

序号	1	2	3	4	5
ai	5	8	7	3	9

记询问区间长度为len, 从左端点向右找一段长为 $2^{\lfloor \log(\text{len}) \rfloor}$ 的区间 (蓝色部分), 右端点向左也找一段长为 $2^{\lfloor \log(\text{len}) \rfloor}$ 的区间 (黄色部分), 显然这两段区间已经覆盖了整个区间 (中间重叠了一块绿色部分), 取最大值即可。

从RMQ问题谈起

查询：若需要查询的区间为 $[i,j]$ ，则需要找到两个覆盖这个闭区间的最小幂区间。

```
lo[0]=-1; //运用递推进行预处理，log以2为底下取整。
for(int k=1;k<=N;k++)
    lo[k]=lo[k>>1]+1;
while(M--){
    scanf("%d%d",&i,&j);
    int k=lo[j-i+1]; //直接调用
    printf("%d\n",max(f[i][k],f[j-(1<<k)+1][k]));
}
```

```
while(M--){
    int i=read(),j=read();
    int k=(int)(log((double)(j-i+1))/log(2.0));
    printf("%d\n",max(f[i][k],f[j-(1<<k)+1][k]));
}
```

从RMQ问题谈起

【总结与思考】

1. **OI竞赛最核心的部分就是观察和利用性质。**善于挖掘隐藏性质，暴力很慢的原因是很多**隐藏的****性质**暴力都没有用，直接抛弃了。如最大值可合并性质。
2. 通过适当地预处理一些区间的最大值，让每一个询问区间都可以用很少的预处理过的区间拼出来。
3. 请同学们呢思考，对于该问题，ST表利用了那些隐藏的性质来完成复杂度的优化呢？
4. 对于这个问题，线段树又可以利用那些隐藏的性质来完成复杂度的优化的？

补充(非严格定义只限于本节课):

可加性: 给出A和B，我们能快速算出 $A+B$ 的值。

可减性: 给出A和B，我们能快速算出 $A-B$ 的值。

可逆性: 给出A和B，始终有 $A+B-B=A$

结合律: $(A+B)+C=A+(B+C)$

作业练习题

Problem 01	P390 [模板]ST表
Problem 02	P391 [SCOI2016]萌萌哒
Problem 03	P392 [USACO07JAN]Balanced Lineup G
Problem 04	P394 Count
Problem 05	P1155 [NOIP2025] 序列询问 (query)
Problem 06	P5998 [BZOJ1012][JSOI2008]区间最值查询模板」最大数maxnumber
Problem 07	P9060 BZOJ 2006 [NOI2010]超级钢琴
Problem 08	P20084 序列
Problem 09	P20549 [2020计蒜客第三套duoxiao1127] T2序列与操作
Problem 10	P1072 [CSP-S 2022] 策略游戏(game)

【CSP-S 2022】策略游戏(game)

【题目描述】

小 L 和小 Q 在玩一个策略游戏。有一个长度为 n 的数组 A 和一个长度为 m 的数组 B ，在此基础上定义一个大小为 $n \times m$ 的矩阵 C ，满足 $C_{ij} = A_i \times B_j$ 。所有下标均从 1 开始。

游戏一共会进行 q 轮，在每一轮游戏中，会事先给出 4 个参数 l_1, r_1, l_2, r_2 ，满足 $1 \leq l_1 \leq r_1 \leq n$ 、 $1 \leq l_2 \leq r_2 \leq m$ 。

游戏中，小 L 先选择一个 $l_1 \sim r_1$ 之间的下标 x ，然后小 Q 选择一个 $l_2 \sim r_2$ 之间的下标 y 。定义这一轮游戏中二人的得分是 C_{xy} 。

小 L 的目标是使得这个得分尽可能大，小 Q 的目标是使得这个得分尽可能小。同时两人都是足够聪明的玩家，每次都会采用最优的策略。

请问：按照二人的最优策略，每轮游戏的得分分别是多少？

【CSP-S 2022】策略游戏(game)

【问题分析】

【样例 #1】

这组数据中，矩阵 C 如下：

	1	2
1	0	0
2	-3	4
3	6	-8

在第一轮游戏中，无论小 L 选取的是 $x = 2$ 还是 $x = 3$ ，小 Q 都有办法选择某个 y 使得最终的得分为负数。因此小 L 选择 $x = 1$ 是最优的，因为这样得分一定为 0。

而在第二轮游戏中，由于小 L 可以选 $x = 2$ ，小 Q 只能选 $y = 2$ ，如此得分为 4。

【CSP-S 2022】策略游戏(game)

【问题分析】

● A 区间全为正数

B 区间全为正数, A 取最大, B 取最小

B 区间有正有负, A 取最小, B 取最小

B 区间全为负数, A 取最小, B 取最小

● A 区间有正有负

B 区间全为正数, A 取最大, B 取最小

B 区间有正有负, $\max(A \text{ 正数最小} \times B \text{ 负数最小}, A \text{ 负数最大} \times B \text{ 正数最大})$

B 区间全为负数, A 取最小, B 取最大

● A 区间全为负数

B 区间全为正数, A 取最大, B 取最大

B 区间有正有负, A 取最大, B 取最大

B 区间全为负数, A 取最小, B 取最大

【CSP-S 2022】策略游戏(game)

【问题分析】 使用 ST 表 (Sparse Table) 来预处理以下 6 种区间最值：

- A 的区间最大值 (amax)
- A 的区间最小值 (amin)
- A 的负数区间最大值 (afmx): 把所有满足 $a_i \geq 0$ 的 a_i 全部替换为 $-\infty$ 。
- A 的非负数区间最小值 (azmn): 把所有满足 $a_i < 0$ 的 a_i 全部替换为 $+\infty$ 。
- B 的区间最大值 (bmax)
- B 的区间最小值 (bmin)

【CSP-S 2022】策略游戏(game)

核心算法:

查询 A 区间 $[l_1, r_1]$ 的 $amax$, $amin$, $afmx$, $azmn$ 。

查询 B 区间 $[l_2, r_2]$ 的 $bmax$, $bmin$ 。

计算四种策略的得分，**取最大值**：

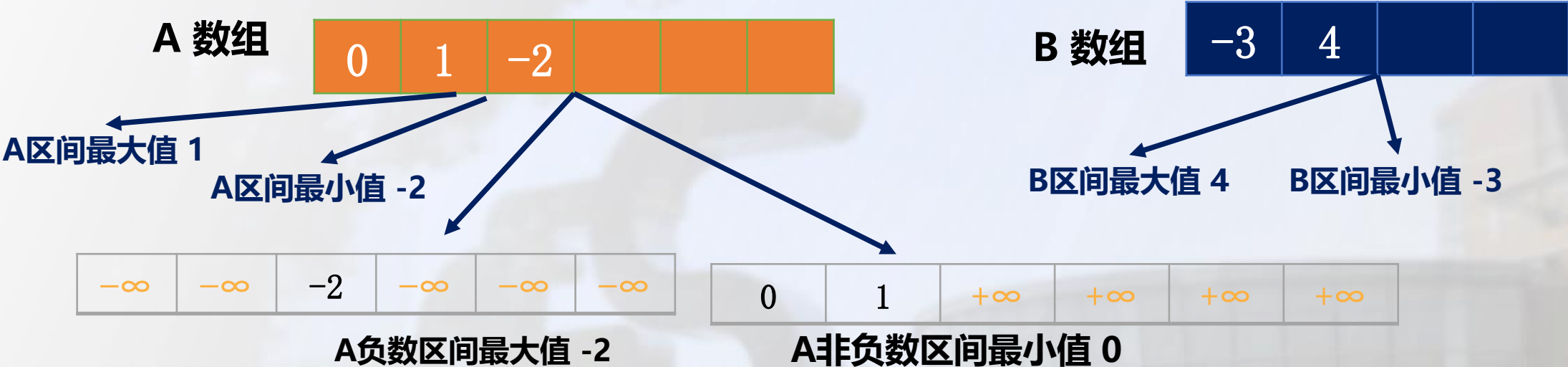
$amax \times (amax \geq 0 ? bmin : bmax)$

$amin \times (amin \geq 0 ? bmin : bmax)$

- 若 $afmx$ 存在（不为 $-\infty$ ），计算 $afmx * (afmx \geq 0 ? bmin : bmax)$ （实际上 $afmx$ 为负，故乘 $bmax$ ）
- 若 $azmn$ 存在（不为 $+\infty$ ），计算 $azmn * (azmn \geq 0 ? bmin : bmax)$ （实际上 $azmn$ 非负，故乘 $bmin$ ）

【CSP-S 2022】策略游戏(game)

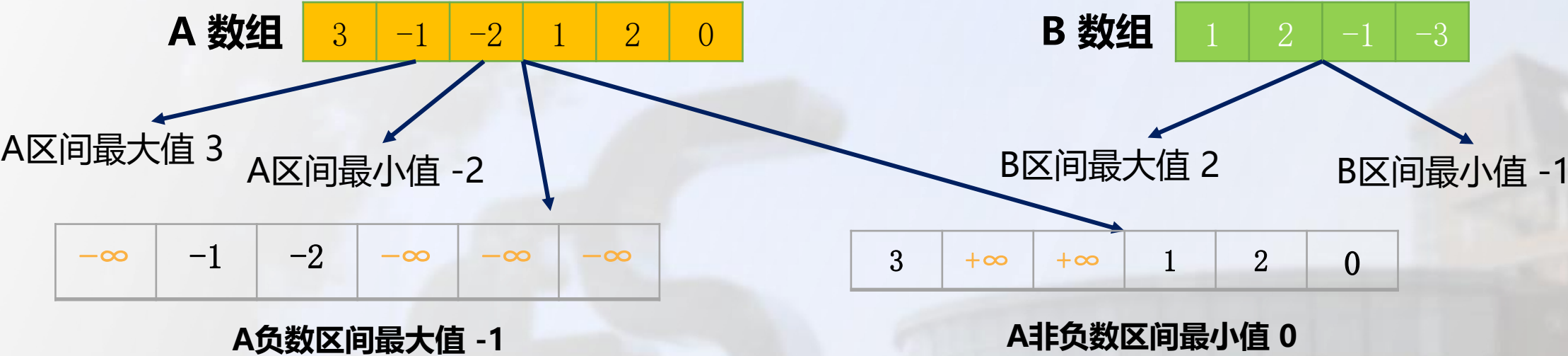
【问题分析】以样例 #1 模拟比较过程:



A 区间最大值 1	B 区间最小值 -3	-3
A 区间最小值 -2	B 区间最大值 4	-8
A 负数区间最大值 -2	B 区间最大值 4	-8
A 非负数区间最小值 0	B 区间最小值 -3	0
取上述结果中的最大值:		0

【CSP-S 2022】策略游戏(game)

【问题分析】 以样例 #2 模拟比较过程:



A 区间最大值 3	B 区间最小值 -1	-3
A 区间最小值 -2	B 区间最大值 2	-4
A 负数区间最大值 -1	B 区间最大值 2	-2
A 非负数区间最小值 0	B 区间最小值 -1	0
取上述结果中的最大值:		0

线段树基础操作

【思考】 什么样的问题可以用线段树和平衡树去维护呢？

答案可加性：支持把这两个子问题A和B的答案快速合并，得到A+B的答案。

合并运算满足结合律： $(A+B)+C=A+(B+C)$

【思考】 如果有区间修改呢？

懒标记

【思考】 具体什么样性质的问题可以使用懒标记来支持区间修改？

懒标记与懒标记具有可加性，且满足结合律。

懒标记与答案也具有可加性，且满足结合律。

线段树基础操作

问题在线

【问题描述】一个长度为 n 的序列 a_n ，我们每次对该数组有一些操作：

- | | | | |
|-----------------|-------------|---------------|-------------|
| 1.修改数组中某个元素的值 | [1,5,4,1,6] | (a[2]=3) | [1,3,4,1,6] |
| 2.询问数组中某段区间的最大值 | [1,5,4,1,6] | (max(1,4)=?) | 5 |
| 3.询问数组中某段区间的和 | [1,5,4,1,6] | (sum(3,5)=?) | 11 |

【问题分析】

如果只有一次询问？

【思路】枚举相应区间内的元素，输出答案 $O(n)$

如果有更多的询问？

【思路】 q 次询问， $O(nq)$ ，主要问题在于询问是针对区间的，而我们维护的信息是针对每个元素的！

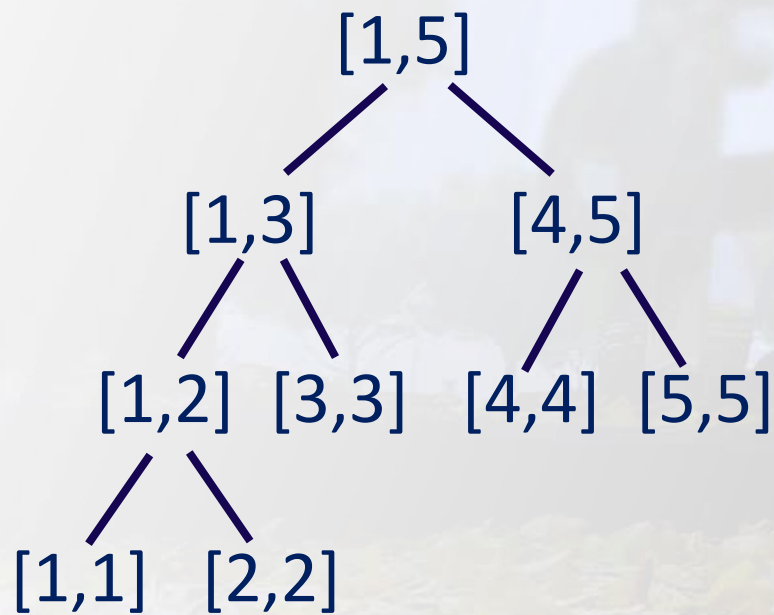
线段树——在 $O(\log_2 n)$ 的时间内完成每次操作。

线段树基础操作

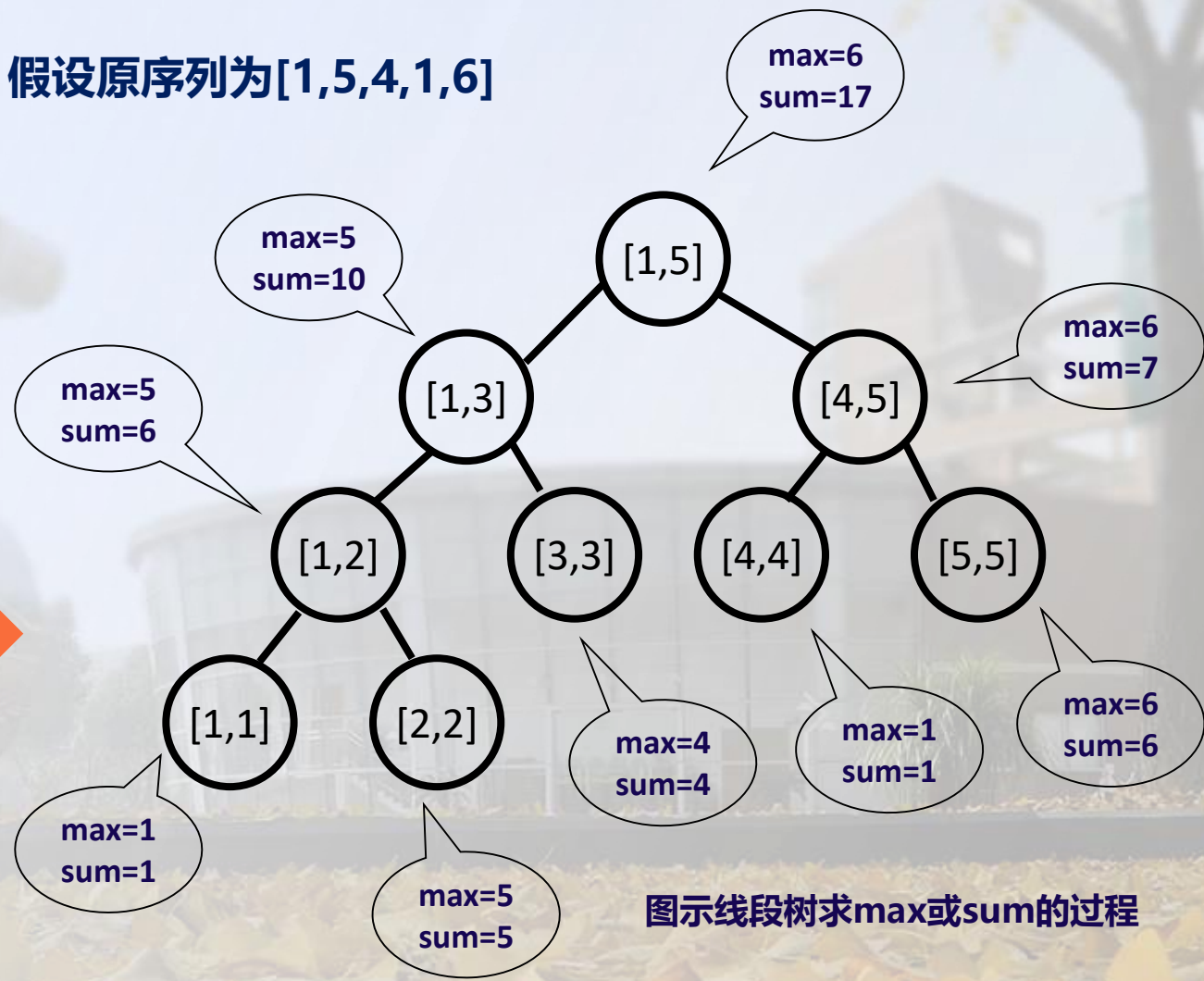
线段树的结构

线段树的本质是一棵二叉树，不同于其它二叉树，线段树的每一个节点记录的是一段区间的信息。

例如：对于长度为5的原数组 $a[1] \sim a[5]$ ，线段树这样存储：



假设原序列为 $[1,5,4,1,6]$



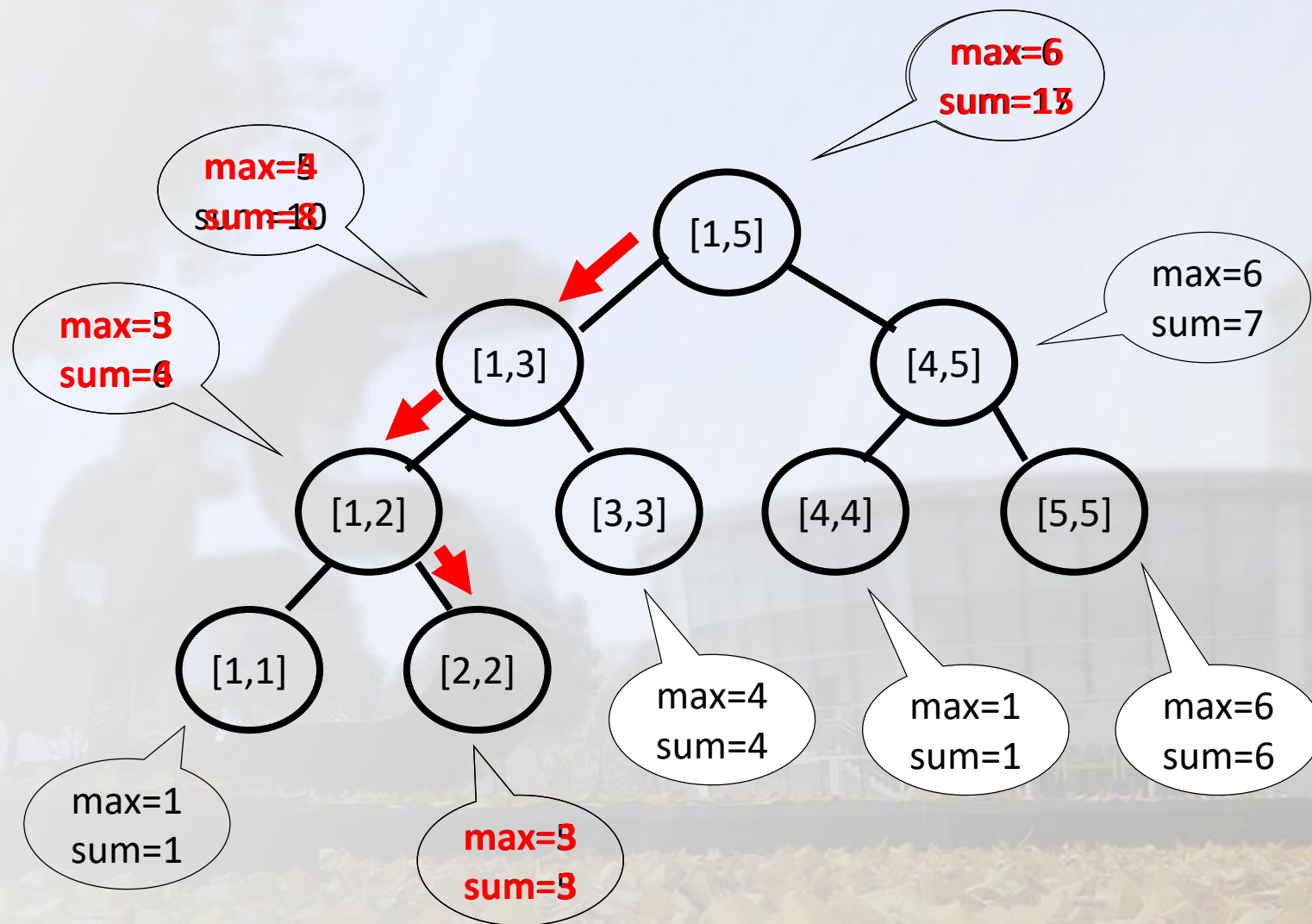
图示线段树求max或sum的过程

线段树基础操作

线段树的结构

更新, $a[2]=3$

图示单点修改的过程



线段树基础操作

首先你要创建树的节点：线段树的每一个节点要记录该节点的左右端点值、区间和、最大值等信息，所以要用结构体数组

如何计算左右儿子的编号？ 怎样迅速计算某个编号为id节点的左右儿子的编号？类似于二叉堆求左右儿子编号。

准备工作做好之后，就可以开始建树啦！

◆ 那么，如果你从头建立一棵线段树，可以这样：

`build(1, 1, n);`

```
const int maxn = 100000;
struct seg_tree{
    int l, r;
    int mx, sum;
}tr[maxn*4];
```

```
#define lid (id<<1)
#define rid (id<<1|1)
```

```
void build(int id, int l, int r){
    tr[id].l = l;
    tr[id].r = r;
    if(l == r){
        tr[id].sum = a[l];
        tr[id].mx = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(lid, l, mid);
    build(rid, mid+1, r);
    tr[id].sum = tr[lid].sum + tr[rid].sum;
    tr[id].mx = max(tr[lid].mx, tr[rid].mx);
}
```


线段树基础操作

单点修改原数组的某个值

单点修改很简单，就是从id开始不断在线段树中往下查找原数组修改的位置，直到找到线段树的叶子节点。即比较修改的位置x与id节点左右端点中点的大小，然后递归往下。

修改后还要往上更新相应线段树节点的信息。

```
void modify(int id, int x, int val){
    if(tr[id].l == tr[id].r){
        tr[id].sum = tr[id].mx = val;
        return;
    }
    int mid = (tr[id].l + tr[id].r) >> 1;
    modify(x <= mid ? lid : rid, x, val);
    tr[id].sum = tr[lid].sum + tr[rid].sum;
    tr[id].mx = max(tr[lid].mx, tr[rid].mx);
}
```

查询区间和

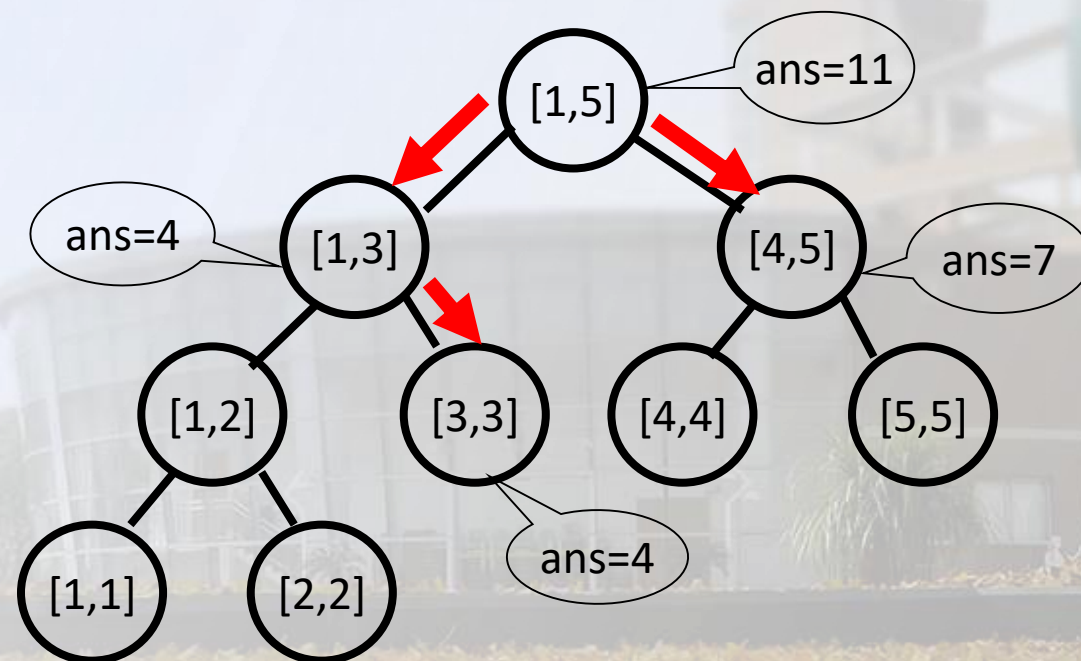
数组[1,5,4,1,6]，以sum(3, 5)为例

查询sum的过程与mx类似，以sum为例

查询sum的区间有**三种情况**：

1. 区间在id节点的左半边，递归往下lid
2. 区间在id节点的右半边，递归往下rid
3. 区间包含id节点的中点，区间以中点分段，分别递归lid、rid

查询的终点应该是递归往下查询的线段树节点的左右端点与递归往下的l、r值相匹配的时候，就可以直接返回具体的值了。



线段树基础操作

区间查询的代码实现

```
int query(int id, int l, int r){
    if(tr[id].l == l && tr[id].r == r)
        return tr[id].sum;
    int mid = (tr[id].l + tr[id].r) >> 1;
    if(r <= mid) return query(lid, l, r);
    if(l > mid) return query(rid, l, r);
    return query(lid, l, mid) + query(rid, mid+1, r);
}
```

如果你要查询原数组某一个区间 l 到 r 的和，调用：**query(1, l, r)**

更新操作：由于总是一条路径从根到某个叶子，而树的深度为 $\log_2 N$ ，因而为 $O(\log_2 N)$ 。

查询操作：每层被访问的节点不超过4个，因而同样为 $O(\log_2 N)$ 。

线段树基础操作

线段树之区间修改

首先，线段树的区间修改不能for区间的每个元素进行单点修改，这样时间复杂度会多一个n。

正确的做法：将标记lazy打在区间上，表示该区间会有一个lazy的增加。

如果在之后的维护或查询过程中，需要对这个结点递归地进行处理，则当场将这个标记分解，传递给它的两个子结点，这样就不需要考虑自根结点开始的影响了。

这种在需要的时候才进行分解的做法，导致了我们的整体处理的时间复杂度仍然能维持在 $O(\log N)$ 的水平上。

与单点修改的线段树类似，只是多了一个lazy标记

```
struct seg_tree{
    int l , r;
    int lazy , sum;
}tr[maxn << 2];
```

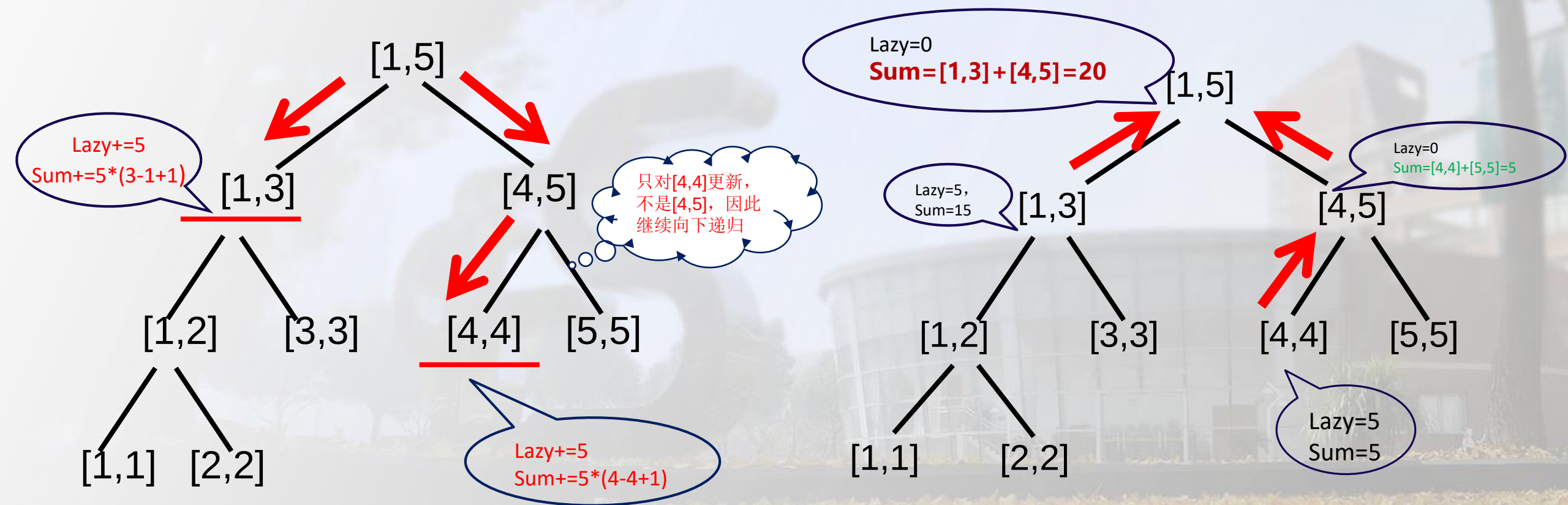
线段树建树

```
void build(int id , int l , int r){
    tr[id].l = l; tr[id].r = r;
    if(l == r){
        tr[id].val = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(lid , l , mid);
    build(rid , mid + 1 , r);
    tr[id].val = tr[lid].val + tr[rid].val;
}
```

线段树基础操作

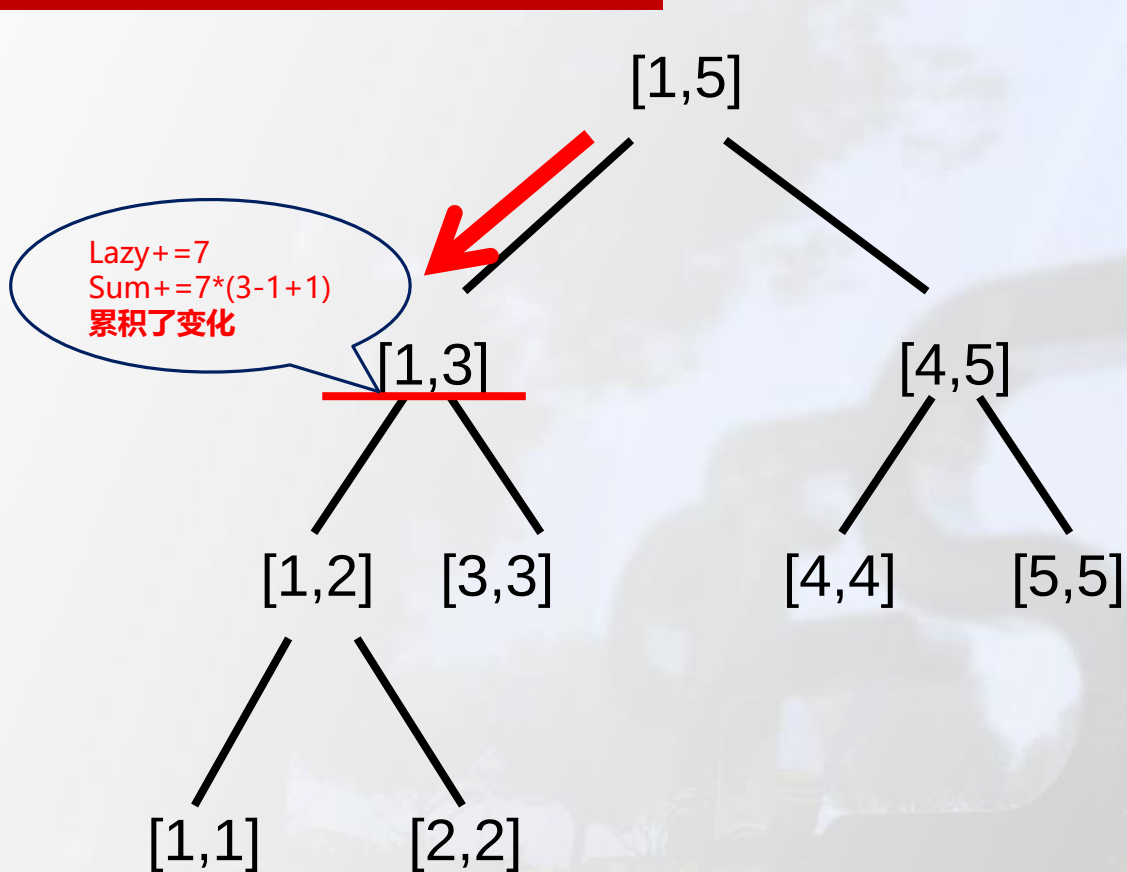
线段树之区间修改

以[1, 4]区间加5为例，第一步：找到区间，打上标记，更新

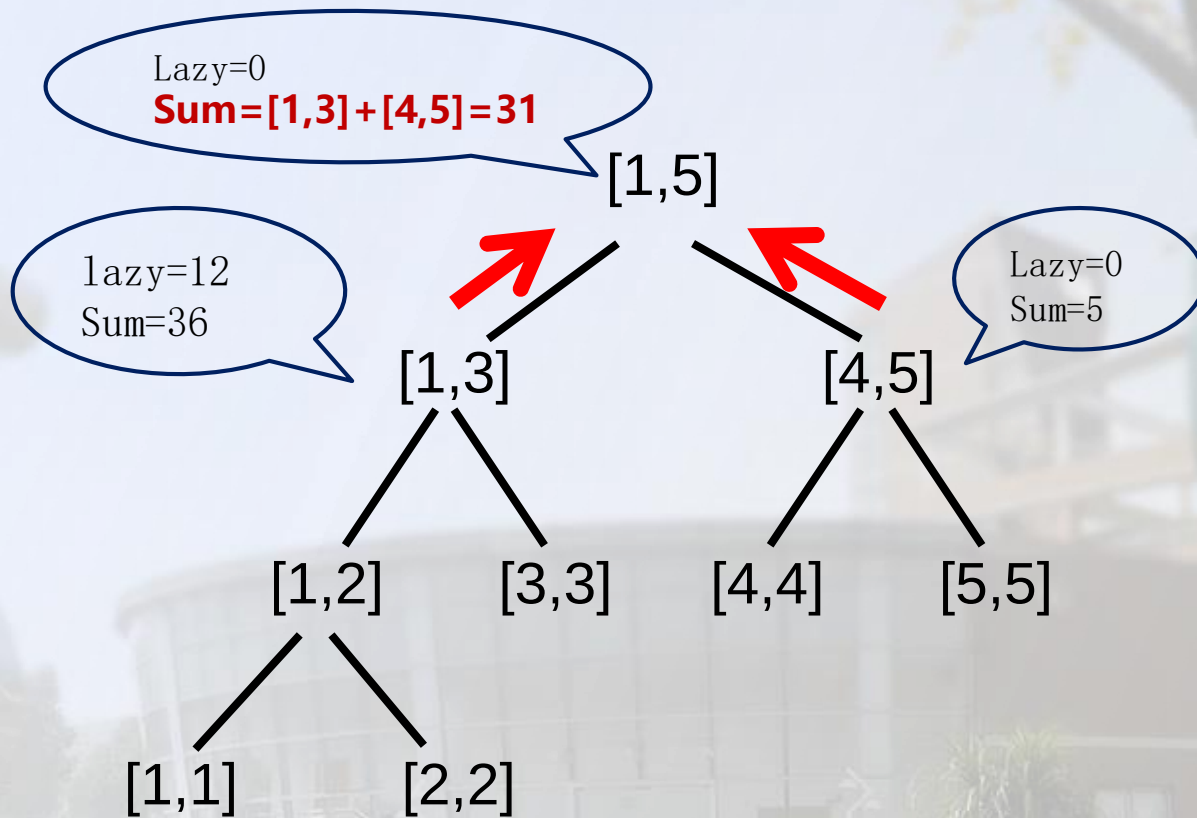


线段树基础操作

线段树之区间修改



如果在上一次打标记后，再来一次区间修改，则会对打过标记的节点累积变化，以 $[1,3]$ 加7为例



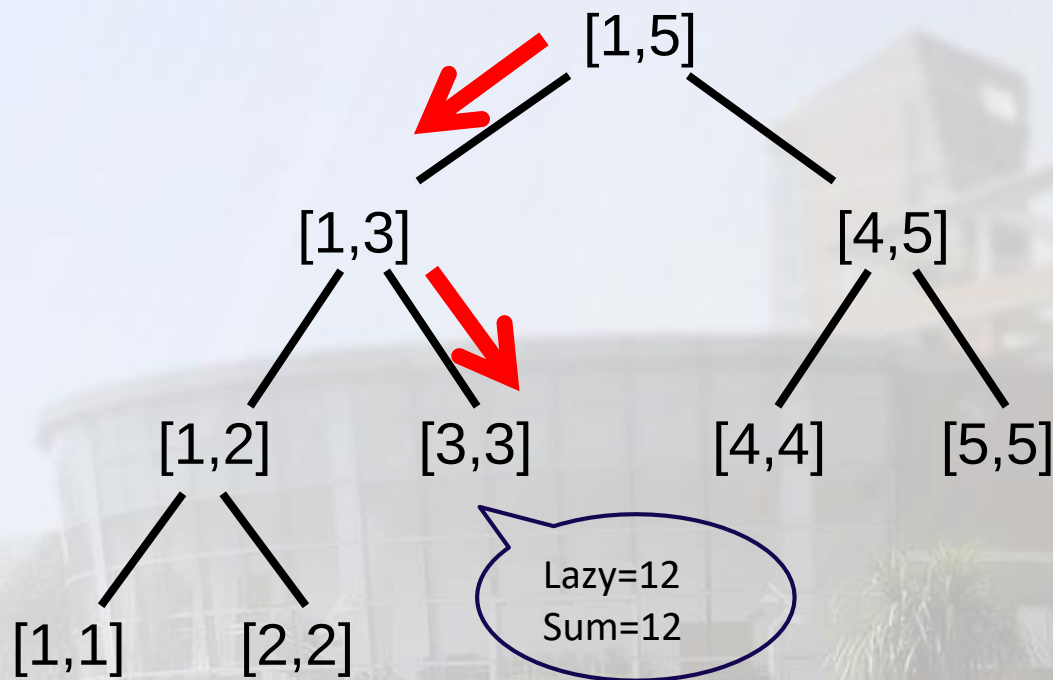
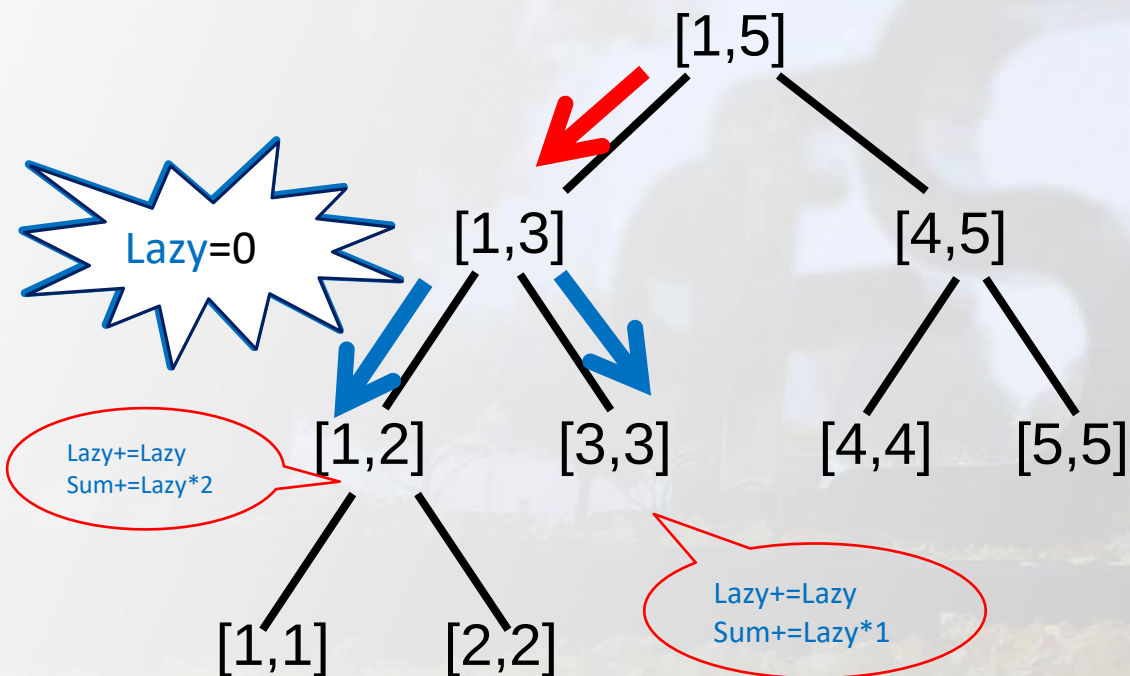
同样，节点信息一旦发生改变，一定要往上更新

线段树基础操作

线段树之区间修改

在查询区间和的时候，与单点修改的线段树一样，首先需要找到查询的区间，不同的是，若递归往下的过程中经过lazy标记不为0的区间，需要下放标记。

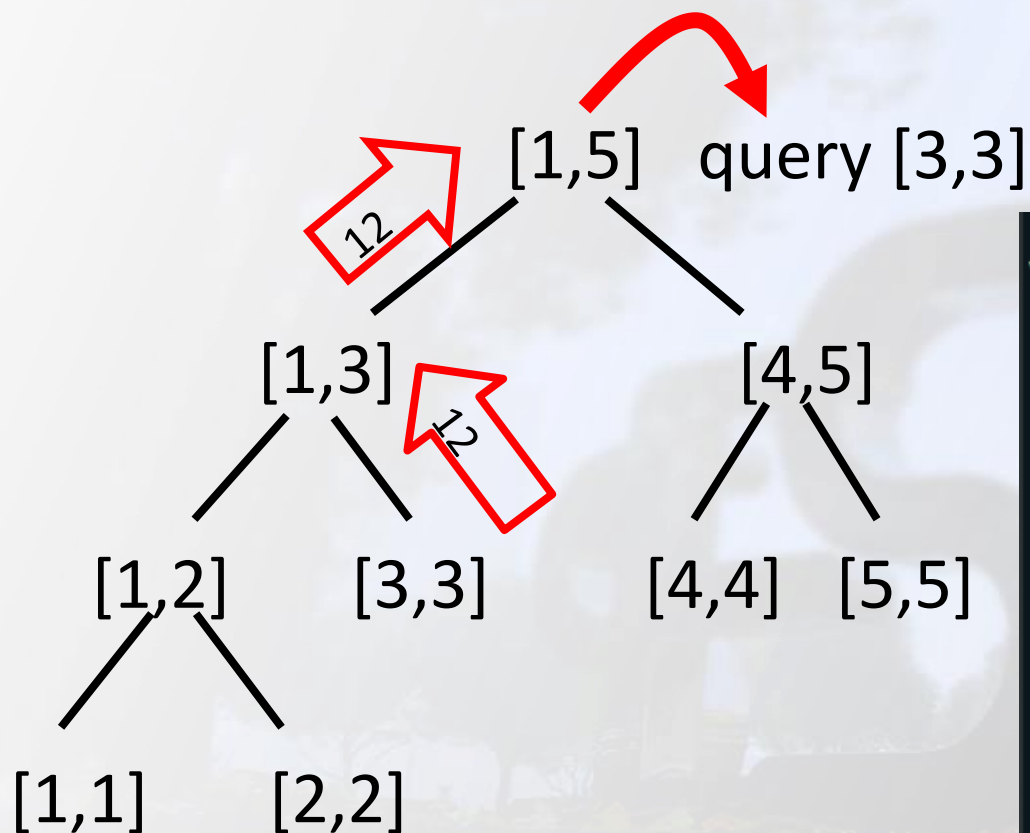
以查询[3,3]区间和为例：



查询区间和的过程

线段树基础操作

线段树之区间修改



查询区间和的过程

```
void add(int id, int l, int r, int val){
    pushdown(id);
    if(tr[id].l == tr[id].r){
        tr[id].lazy += val;
        tr[id].sum += val*(tr[id].r - tr[id].l + 1);
        return;
    }
    int mid = (l + r) >> 1;
    if(r <= mid) add(lid, l, r, val);
    else if(l > mid) add(rid, l, r, val);
    else add(lid, l, mid, val), add(rid, mid+1, r, val);
    tr[id].sum = tr[lid].sum + tr[rid].sum;
}
```

区间修改代码实现

线段树基础操作

线段树之区间修改

下放标记参考代码

```
void pushdown(int id){
    if(tr[id].lazy && tr[id].l != tr[id].r){
        tr[lid].lazy += tr[id].lazy;
        tr[rid].lazy += tr[id].lazy;
        tr[lid].sum += tr[id].lazy * (tr[lid].r - tr[lid].l + 1);
        tr[rid].sum += tr[id].lazy * (tr[rid].r - tr[rid].l + 1);
        tr[id].lazy = 0;
    }
}
```

询问区间和参考代码

```
int query(int id , int l , int r){
    pushdown(id);
    if(tr[id].l == l && tr[id].r == r) return tr[id].sum;
    int mid = (tr[id].l + tr[id].r) >> 1;
    if(mid >= r) return query(lid , l , r);
    else if(mid < l) return query(rid , l , r);
    else return query(lid , l , mid) + query(rid , mid + 1 , r);
}
```


线段树基础操作

【试一试】 请同学们判断下面的问题，能否使用线段树/平衡树来维护。

问题1： 有一个序列，要求支持：修改一个元素；查询区间和。

可以，“子序列和”这个问题显然具有可加性和结合律。

问题2： 有一个序列，要求支持：修改一个元素；查询区间前缀最大和。

可以，维护和以及前缀最大和这两个值即可支持合并

问题3： 有一个序列，要求支持：区间整体加一个值；查询区间和。

可以，以“这个区间被和加了多少”作为懒标记，显然懒标记内部、懒标记和答案之间都具有可加性和结合律。

问题4： 有一个序列，要求支持：区间整体加一个值；查询区间前缀最大和。

不可以，难以快速合并答案

线段树基础操作

【试一试】 请同学们判断下面的问题，能否使用线段树/平衡树来维护。

问题5： 有一个序列，要求支持：修改一个元素；查询区间平方和。

可以，“子序列的平方和”显然满足可加性和结合律。

问题6： 有一个序列，要求支持：区间整体增加一个值，查询区间平方和。

可以，维护“子序列的平方和”和“子序列和”这两个值，可以发现这满足懒标记的应用条件。

问题7： 有一个序列，要求支持：修改一个元素；查询区间最大公约数。

可以，“子序列的最大公约数”显然满足可加性和结合律。

问题8： 有一个序列，要求支持：区间整体加一个值，查询区间最大公约数。

可以，虽然直接维护的话，懒标记确实是无法快速更新答案的；但如果我们维护邻项做差后的序列，那么区间修改就变成了单点修改，而邻项做差是不会影响最大和公约数的结果的。

线段树基础操作

【试一试】 请同学们判断下面的问题，能否使用线段树/平衡树来维护。

问题9：有一个序列，要求支持：查询第K大数。

可以，虽然直接做很困难，但只需先二分答案，问题便转化为了“一个区间中有多少个数大于某个值”，这个问题只需记录区间中的数排序后的结果即可快速支持查询。

问题10：有一个序列，要求支持：修改一个元素，查询区间第K大数。

可以，与上题类似，先二分答案，问题便转化为了“一个区间中有多少个数大于某个值”，我们需要维护区间中的数排序后的结果，并支持修改一个元素。因此用平衡树维护区间中的数排序后的结果即可。

问题11：有一个序列，要求支持：区间整体加一个值；查询区间的第k大数。

不可以，难以快速合并答案。

问题12：有一个序列，要求支持：查询区间众数。

不可以，难以快速合并答案。

思考：没有可加性（无法快速合并答案）怎么办？ **分块结构、离散化处理等转化。**

线段树基础操作

【题目01】（洛谷P3373）有一个长度为 n 的数列，在这个数列上进行 m 次操作：

操作1：1 x y k 含义：将区间 $[x, y]$ 内每个数**乘上** k

操作2：2 x y k 含义：将区间 $[x, y]$ 内每个数**加上** k

操作3：3 x y 含义：输出区间 $[x, y]$ 内每个数的和

【分析】使用线段树，每个节点维护对应区间所有数的和 s 。设数组一开始全为1，那么线段树如下图所示：

S=16															
S=8								S=8							
S=4				S=4				S=4				S=4			
S=2		S=2		S=2		S=2		S=2		S=2		S=2		S=2	
S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

【分析】假设接下来执行修改2 4 12 1，则区间[4,12]中的每个位置都加上1，s的变化如图。

S=25															
S=13								S=12							
S=5				S=8				S=8				S=4			
S=2		S=3		S=4		S=4		S=4		S=4		S=2		S=2	
S=1	S=1	S=1	S=2	S=2	S=2	S=2	S=2	S=2	S=2	S=2	S=2	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

【分析】但是需要修改的节点太多了，为了保证效率，我们改为修改 $\log n$ 个节点的信息，并在这些节点上打上**延迟标记**。

S=25															
S=13								S=12							
S=5				S=8 +1				S=8 +1				S=4			
S=2		S=3		S=2		S=2		S=2		S=2		S=2		S=2	
S=1	S=1	S=1	S=2 +1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

此时标记下方的节点维护的信息是不真实的，但没关系，我们可以通过**下传标记(Pushdown)**来得到真实信息。也就是说等我们下次要访问下方的节点时再把标记往下传，获取真实的信息即可。

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

【分析】假设接下来执行操作1 11 12 3，由于标记讲究先来后到，所以在访问到节点x之前，我们要把根到x父亲这条路上的标记都做一次下传：

S=25															
S=13								S=12							
S=5				S=8 +1				S=8				S=4			
S=2		S=3		S=2		S=2		S=4 +1		S=4 +1		S=2		S=2	
S=1	S=1	S=1	S=2 +1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

这样，从x的父亲到根的节点都没有标记了，节点x的信息就是真实的。我们可以放心地更新节点x的信息，并在节点x处打上乘标记

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

【分析】假设接下来执行操作1 11 12 3，由于标记讲究先来后到，所以在访问到节点x之前，我们要把根到x父亲这条路上的标记都做一次下传：

S=25															
S=13								S=12							
S=5				S=8 +1				S=8				S=4			
S=2		S=3		S=2		S=2		S=4 +1		S=12 *3+3 乘		S=2		S=2	
S=1	S=1	S=1	S=2 +1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

这样，从x的父亲到根的节点都没有标记了，节点x的信息就是真实的。我们可以放心地更新节点x的信息，并在节点x处打上乘标记

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

【分析】最后，我们需要从下往上更新一遍节点的信息：

S=33															
S=13								S=20							
S=5				S=8 ⁺¹				S=16				S=4			
S=2		S=3		S=2		S=2		S=4 ⁺¹		S=12 ^{*3+3}		S=2		S=2	
S=1	S=1	S=1	S=2 ⁺¹	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1	S=1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

修改操作：

```
void update(int l,int r,int c,int s,int t,int p){
    //[l,r]为修改区间，c为被修改的元素的变化量，
    //[s,t]为当前节点包含的区间，p为当前节点的编号
    if(l<=s && t<=r){
        d[p]+=(t-s+1)*c,b[p]+=c;
        return;
    } // 当前区间为修改区间的子集时直接修改当前节点的值，然后打标记，结束修改。
    int m=s+((t-s)>>1);
    if(b[p]&&s!=t){
        //如果当前节点的懒标记非空，则更新当前节点两个子节点的值和懒标记
        d[p*2]+=b[p]*(m-s+1);
        d[p*2+1]+=b[p]*(t-m);
        b[p*2]+=b[p];
        b[p*2+1]+=b[p]; //将标记下传给子节点
        b[p]=0; //清空当前节点的标记。
    }
    if(l<=m) update(l,r,c,s,m,p*2);
    if(r>m) update(l,r,c,m+1,t,p*2+1);
    d[p]=d[p*2]+d[p*2+1];
}
```

在访问子节点之前需要进行**PushDown**操作，**PushDown**需要完成：

1. 获取两个子节点真实的信息
2. 下传标记，跟子节点的标记合并

PushUp需要根据两个子节点的信息得到当前节点的信息

线段树基础操作

【题目01】（洛谷P3373）有一个长度为n的数列，在这个数列上进行m次操作：

查询操作类似，

只是不需要

PushUp操作：

```
void getsum(int l,int r,int s,int t,int p) {
    //[l,r]为查询区间，[s,t]为当前包含的区间，p为当前节点的编号。
    if(l<=s && t<=r ) return d[p];
    //当前区间为询问区间的子集时直接返回当前区间的和
    int m=s+((t-s)>>1);
    if(b[p]){
        //如果当前节点的懒标记非空
        //则更新当前节点两个子节点的值和懒标记值
        d[p*2]+=b[p]*(m-s+1);
        d[p*2+1]+=b[p]*(t-m);
        b[p*2]+=b[p];
        b[p*2+1]+=b[p]; //将标记下传给子节点
        b[p]=0; //清空当前节点的标记
    }
    int sum=0;
    if(l<=m) sum=getsum(l,r,s,m,p*2);
    if(r>m) sum+=getsum(l,r,m+1,t,p*2+1);
    return sum;
}
```

线段树基础操作

作业练习题

Problem 1	P514 [线段树模板题]--序列操作1
Problem 2	P515 [线段树模板题]--序列操作2
Problem 3	P516 农场分配
Problem 4	P518 [AHOI2009]--维护序列-区间乘与区间加
Problem 5	P519 [树的dfs序]--工资查询
Problem 6	P521 Can you answer on these queries III
Problem 7	P6182 「算阶」「0X40数据结构进阶」A Simple Problem with Integers (线段树)
Problem 8	P1832 [USACO08FEB] Hotel G
Problem 9	P1833 [USACO08NOV] Light Switching G